LAPORAN

PEMROGRAMAN BERORIENTASI OBJEK

SISTEM PERPUSTAKAAN



Disusun Oleh:

AULIA SHEVA SAVITRI	2509116001
MUHAMMAD YUSRI JIBRAN S.J.	2509116030
JIHAN SHAFIRA RAHMAH	2509116073
OTNIEL PUTRA WARDANA	2509116081

PROGRAM STUDI SISTEM INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS MULAWARMAN

2026

DAFTAR ISI

DAFTAR ISI i
DAFTAR GAMBAR ii
DAFTAR TABEL iii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Deskripsi Studi Kasus 2
BAB II IMPLEMENTASI PROGRAM 3
2.1 Penerapan Struktur Data 3
2.1.1 Array 1 Dimensi 3
2.1.2 Array 2 Dimensi 3
2.1.3 Array 3 Dimensi 3
2.1.4 ArrayList 3
2.1.5 Stack (LIFO) 4
2.1.6 Queue (FIFO) 4
2.2 Struktur dan Implementasi Source Code 4
2.3 Penjelasan Source Code 5
2.3.1 Penjelasan MainPerpustakaan.java 5
2.3.2 Penjelasan DataPeminjaman.java 11
2.3.1 Penjelasan ManajemenBuku.java 20
2.3.1 Penjelasan TransaksiAntrean.java 39
2.4 Tampilan dan Hasil Eksekusi Program 51
BAB III PENGUJIAN DAN PEMBAHASAN 61
3.1 Hasil Pengujian Program 61
3.2 Analisis Fungsionalitas Program 62
BAB IV PENUTUP 63
4.1 Kesimpulan 63

DAFTAR GAMBAR

Gambar 2. 1 Tampilan Awal dan Input Nama Petugas 51
Gambar 2. 2 Tampilan Menu Utama Sistem Manajemen Perpustakaan Kampus 51
Gambar 2. 3 Tampilan Daftar Kategori Buku pada Sistem Manajemen Perpustakaan Kampus 52
Gambar 2. 4 Tampilan Data Peminjaman Buku Mingguan Berdasarkan Kategori dan Hari 52
Gambar 2. 5 Tampilan Proses Perubahan Data Peminjaman Buku 53
Gambar 2. 6 Tampilan Total Peminjaman Buku Berdasarkan Kategori 53
Gambar 2. 7 Tampilan Lokasi Penyimpanan dan Status Ketersediaan Buku 54
Gambar 2. 8 Tampilan Lokasi Penyimpanan dan Status Ketersediaan Buku Lantai 2 54
Gambar 2. 9 Tampilan Proses Pemilihan Lokasi Penyimpanan Buku 55
Gambar 2. 10. Tampilan Konfirmasi Penambahan Buku Berhasil 55
Gambar 2. 11 Tampilan Daftar Koleksi Buku dan Status Ketersediaan Mahasiswa 56
Gambar 2. 12 Tampilan Hasil Pencarian Buku dan Informasi Lokasi Penyimpanan 56
Gambar 2. 13 Tampilan Proses Penghapusan Buku dari Koleksi Perpustakaan 56
Gambar 2. 14 Tampilan Proses Penambahan Mahasiswa ke Antrean Peminjaman Buku 57
Gambar 2. 15 Tampilan Daftar Antrean Mahasiswa Peminjaman Buku 57
Gambar 2. 16 Tampilan Proses Pelayanan Antrean dan Input Data Peminjaman Buku 58
Gambar 2. 17 Tampilan Konfirmasi Keberhasilan Transaksi Peminjaman Buku 58
Gambar 2. 18 Tampilan Riwayat Transaksi Peminjaman Buku 59
Gambar 2. 19 Tampilan Informasi Transaksi Peminjaman Terakhir 59
Gambar 2. 20 Tampilan Proses Pembatalan Transaksi Peminjaman Buku 59
Gambar 2. 21 Tampilan Penutupan Program Sistem Manajemen Perpustakaan Kampus 60

DAFTAR TABEL

Tabel 2. 1 Nama Class dan Fungsi pada Sistem Manajemen Perpustakaan Kampus 4
Tabel 3.1 Hasil Pengujian Program……..61

BAB I
PENDAHULUAN

1.1 Latar Belakang

Perpustakaan kampus merupakan salah satu fasilitas yang mendukung kegiatan belajar mahasiswa melalui penyediaan berbagai koleksi buku. Pengelolaan perpustakaan membutuhkan pencatatan yang teratur agar petugas dapat mengetahui daftar buku, lokasi penyimpanan, jumlah peminjaman, serta antrean mahasiswa yang ingin meminjam buku. Pengelolaan data yang kurang terstruktur dapat menyulitkan petugas dalam mencari informasi dan memantau proses peminjaman.

Bahasa pemrograman Java dapat digunakan untuk membantu pengelolaan data perpustakaan agar lebih teratur dan mudah diakses. Java menyediakan berbagai struktur data, seperti Array 1 Dimensi, Array 2 Dimensi, Array 3 Dimensi, ArrayList, Stack, dan Queue, yang memiliki fungsi berbeda sesuai kebutuhan. Struktur data tersebut dapat dimanfaatkan untuk menyimpan kategori buku, mencatat jumlah peminjaman, mengatur lokasi penyimpanan, mengelola koleksi buku, serta memproses antrean dan riwayat transaksi.

Sistem Manajemen Perpustakaan Kampus dikembangkan sebagai program sederhana berbasis Java yang mengintegrasikan keenam struktur data tersebut dalam satu aplikasi. Program ini menyediakan menu interaktif bagi petugas untuk mengelola koleksi dan lokasi buku, memperbarui data peminjaman, serta mengatur antrean mahasiswa dan riwayat transaksi. Penerapan program ini diharapkan dapat meningkatkan pemahaman mengenai penggunaan struktur data dalam menyelesaikan permasalahan pengelolaan informasi secara terstruktur.

1.2 Tujuan

Pembuatan program Sistem Manajemen Perpustakaan Kampus bertujuan untuk:

1. Menerapkan enam struktur data dalam bahasa pemrograman Java, yaitu Array 1 Dimensi, Array 2 Dimensi, Array 3 Dimensi, ArrayList, Stack, dan Queue.
2. Mempermudah pengelolaan koleksi buku, lokasi penyimpanan, dan data peminjaman berdasarkan kategori serta hari.

3. Mengelola antrean mahasiswa berdasarkan urutan kedatangan serta menyimpan riwayat transaksi menggunakan prinsip LIFO.
4. Menerapkan menu interaktif dan penanganan kesalahan (error handling) agar program dapat digunakan dengan lebih mudah dan mengurangi kesalahan input.
5. Memahami cara mengintegrasikan berbagai struktur data ke dalam satu program yang memiliki fungsi saling berkaitan.

1.3 Deskripsi Studi Kasus

Perpustakaan kampus membutuhkan sistem sederhana untuk mengelola koleksi buku, lokasi penyimpanan, dan transaksi peminjaman mahasiswa. Petugas perlu mengetahui ketersediaan buku, jumlah peminjaman berdasarkan kategori dan hari, serta urutan mahasiswa yang akan dilayani. Riwayat transaksi juga perlu dicatat agar transaksi terakhir dapat diperiksa atau dibatalkan apabila diperlukan.

Permasalahan tersebut menjadi dasar pengembangan Sistem Manajemen Perpustakaan Kampus menggunakan Java. Program menerapkan enam struktur data, yaitu Array 1 Dimensi, Array 2 Dimensi, Array 3 Dimensi, ArrayList, Stack, dan Queue, untuk menangani kebutuhan pengelolaan data tersebut melalui menu interaktif.

BAB II
IMPLEMENTASI PROGRAM

2.1 Penerapan Struktur Data

Program Sistem Manajemen Perpustakaan Kampus dibuat menggunakan bahasa pemrograman Java dengan menerapkan enam jenis struktur data. Setiap struktur data memiliki fungsi yang berbeda sesuai kebutuhan pengelolaan buku dan transaksi peminjaman. Penerapan struktur data tersebut dijelaskan sebagai berikut.

2.1.1 Array 1 Dimensi

Array 1 Dimensi digunakan untuk menyimpan daftar kategori buku dan nama hari. Data kategori meliputi Pemrograman, Matematika, Jaringan Komputer, dan Basis Data. Data hari digunakan sebagai acuan pencatatan jumlah peminjaman selama satu minggu.

2.1.2 Array 2 Dimensi

Array 2 Dimensi digunakan untuk menyimpan jumlah peminjaman buku berdasarkan kategori dan hari. Struktur ini memungkinkan petugas menampilkan, memperbarui, dan menghitung total peminjaman pada setiap kategori buku.

2.1.3 Array 3 Dimensi

Array 3 Dimensi digunakan untuk merepresentasikan lokasi penyimpanan buku berdasarkan tiga tingkat, yaitu lantai, rak, dan posisi buku. Struktur ini membantu petugas mengetahui lokasi buku serta menentukan tempat penyimpanan ketika menambahkan koleksi baru.

2.1.4 ArrayList

ArrayList digunakan untuk mengelola daftar koleksi buku secara dinamis. Fitur yang tersedia meliputi menambahkan buku, menampilkan seluruh koleksi, mencari buku, dan menghapus buku yang memenuhi ketentuan.

2.1.5 Stack (LIFO)

Stack digunakan untuk menyimpan riwayat transaksi peminjaman dengan prinsip Last In, First Out (LIFO), yaitu transaksi yang terakhir masuk menjadi transaksi pertama yang diambil. Struktur ini digunakan untuk menampilkan transaksi terakhir dan membatalkan transaksi terbaru.

2.1.6 Queue (FIFO)

Queue digunakan untuk mengelola antrean mahasiswa yang ingin meminjam buku dengan prinsip First In, First Out (FIFO), yaitu mahasiswa yang datang lebih dahulu akan dilayani lebih dahulu. Antrean diproses berdasarkan urutan kedatangan mahasiswa.

2.2 Struktur dan Implementasi Source Code

Program dikembangkan menggunakan Apache NetBeans dengan bahasa pemrograman Java. Source code dibagi menjadi empat class agar fungsi pengelolaan data, koleksi buku, transaksi, dan menu utama tersusun lebih teratur.

Tabel 2. 1 Nama Class dan Fungsi pada Sistem Manajemen Perpustakaan Kampus

Nama Class	Fungsi
MainPerpustakaan.java	Menampilkan menu utama, menerima pilihan petugas, dan menghubungkan fitur program.
DataPeminjaman.java	Mengelola kategori buku, data peminjaman mingguan, dan perhitungan jumlah peminjaman.
ManajemenBuku.java	Mengelola koleksi buku, lokasi penyimpanan, pencarian, penambahan, dan penghapusan buku.
TransaksiAntrean.java	Mengelola antrean mahasiswa, proses peminjaman, serta riwayat dan pembatalan transaksi.

Keempat class tersebut berada dalam package sistemperpustakaan dan saling terhubung melalui pemanggilan method. Program menggunakan Scanner untuk menerima masukan pengguna serta menerapkan validasi dan error handling untuk menangani input yang tidak sesuai.

2.3 Penjelasan Source Code

2.3.1 Penjelasan MainPerpustakaan.java

```
package sistemperpustakaan;
import java.util.Scanner;
```

Package sistemperpustakaan; digunakan untuk menunjukkan bahwa class termasuk dalam package sistemperpustakaan, sedangkan import java.util.Scanner; digunakan untuk mengimpor class Scanner agar program dapat menerima input dari pengguna melalui keyboard.

```
public static void main(String[] args) {

    Scanner input = new Scanner(System.in);

    int pilihan = -1;
```

Public static void main(String[] args) merupakan method utama yang pertama kali dijalankan saat program dimulai. Scanner input = new Scanner(System.in); digunakan untuk menerima input dari pengguna melalui keyboard, sedangkan int pilihan = -1; digunakan untuk menyimpan pilihan menu yang dimasukkan oleh petugas dengan nilai awal -1.

```
System.out.println("==================================");

    System.out.println("SISTEM MANAJEMEN PERPUSTAKAAN");

    System.out.println("==================================");

    String nama;

    do {
```

```
        System.out.print("Masukkan nama petugas: ");
        nama = input.nextLine().trim();

        if (nama.isEmpty()) {
            System.out.println(
                "ERROR: Nama petugas tidak boleh kosong!");
        }
    } while (nama.isEmpty());
```

Bagian ini digunakan untuk menampilkan judul program, kemudian meminta petugas memasukkan nama sebelum mengakses menu utama. Nama yang dimasukkan disimpan dalam variabel nama, sedangkan trim() menghapus spasi di awal dan akhir input. Kondisi if memeriksa apakah nama kosong, lalu menampilkan pesan kesalahan jika belum diisi. Perulangan do-while memastikan petugas harus memasukkan nama yang valid sebelum program melanjutkan ke tahap berikutnya.

```
System.out.println("\nSelamat datang, " + nama + "!");
        System.out.println("Login sebagai Petugas Perpustakaan.");
        do {
            System.out.println("\n================================");
            System.out.println(" MENU UTAMA PERPUSTAKAAN KAMPUS");
            System.out.println("================================");
            System.out.println("1. Tampilkan Kategori Buku");
            System.out.println("2. Data Peminjaman Mingguan");
            System.out.println("3. Ubah Data Peminjaman");
            System.out.println("4. Total Peminjaman");
            System.out.println("5. Tampilkan Lokasi Buku");
            System.out.println("6. Tambah Buku");
            System.out.println("7. Tampilkan Koleksi Buku");
            System.out.println("8. Cari Buku");
```

```
        System.out.println("9. Hapus Buku");
        System.out.println("10. Tambah Antrean Mahasiswa");
        System.out.println("11. Tampilkan Antrean");
        System.out.println("12. Layani Antrean / Pinjam Buku");
        System.out.println("13. Riwayat Transaksi");
        System.out.println("14. Lihat Transaksi Terakhir");
        System.out.println("15. Batalkan Transaksi Terakhir");
        System.out.println("0. Keluar");
        System.out.println("==================================");
```

Bagian ini digunakan untuk menampilkan pesan selamat datang sesuai nama petugas yang telah dimasukkan, kemudian menampilkan menu utama yang berisi 15 pilihan fitur dan opsi keluar. Setiap pilihan mewakili fungsi tertentu, seperti pengelolaan kategori dan koleksi buku, pencatatan peminjaman, pengaturan antrean mahasiswa, serta pengelolaan riwayat transaksi. Perulangan do-while memastikan menu ditampilkan kembali setelah suatu fitur selesai dijalankan, sehingga petugas dapat menggunakan fitur lain tanpa harus memulai ulang program.

```
System.out.print("Pilih menu (0-15): ");
        try {
            pilihan = Integer.parseInt(
                input.nextLine().trim());
            switch (pilihan) {
                case 1:
                 DataPeminjaman.tampilKategori();
                    break;
                case 2:
                    DataPeminjaman.tampilPeminjaman();
```

```
            break;
        case 3:
            DataPeminjaman.ubahPeminjaman(input);
            break;
        case 4:
            DataPeminjaman.totalPeminjaman();
            break;
        case 5:
            ManajemenBuku.tampilLokasi();
            break;
        case 6:
            ManajemenBuku.tambahBuku(input);
            break;
        case 7:
            ManajemenBuku.tampilBuku();
            break;
        case 8:
            ManajemenBuku.cariBuku(input);
            break;
        case 9:
            ManajemenBuku.hapusBuku(input);
```

```
            break;
        case 10:
            TransaksiAntrean.tambahAntrean(input);
            break;
        case 11:
            TransaksiAntrean.tampilAntrean();
            break;
        case 12:
            TransaksiAntrean.layaniAntrean(input);
            break;
        case 13:
            TransaksiAntrean.tampilRiwayat();
            break;
        case 14:
            TransaksiAntrean.transaksiTerakhir();
            break;
        case 15:
            TransaksiAntrean.batalkanTransaksi();
            break;
```

Bagian ini digunakan untuk menerima pilihan menu dari petugas dan menjalankan fitur sesuai angka yang dimasukkan. Perintah try menangani proses pembacaan input, sedangkan Integer.parseInt() mengubah input menjadi bilangan bulat. Percabangan switch-

case menentukan fitur yang dijalankan, mulai dari pengelolaan kategori dan koleksi buku hingga antrean serta riwayat transaksi. Setiap case memanggil fungsi dari class yang sesuai, sementara break menghentikan proses pada pilihan tersebut agar tidak berlanjut ke pilihan berikutnya.

```
case 0:
                System.out.println(
                   "\nTerima kasih, " + nama + "!");
                System.out.println(
                   'Program perpustakaan ditutup.");
                break;
```

Bagian ini digunakan untuk mengakhiri penggunaan program ketika petugas memilih menu 0. Program menampilkan pesan terima kasih dengan menyertakan nama petugas, kemudian menampilkan informasi bahwa program perpustakaan ditutup. Perintah break menghentikan proses pada pilihan tersebut, sehingga program dapat keluar dari perulangan menu utama.

```
default:
                System.out.println(
                   "ERROR: Pilihan menu tidak tersedia!");
            }
        } catch (NumberFormatException e) {
            System.out.println(
              "ERROR: Pilihan harus berupa angka bulat!");
        }
```

Bagian ini digunakan untuk menangani pilihan menu yang tidak tersedia dan kesalahan input dari pengguna. Bagian default menampilkan pesan kesalahan jika angka yang dimasukkan tidak sesuai dengan pilihan menu, sedangkan catch (NumberFormatException e) menangani input yang bukan berupa angka bulat. Dengan adanya penanganan kesalahan ini, program dapat menampilkan pesan yang sesuai tanpa langsung berhenti ketika pengguna memasukkan input yang tidak valid.

```
} while (pilihan != 0);

        input.close();

    }

}
```

Bagian ini digunakan untuk mengulang tampilan menu utama selama variabel pilihan tidak bernilai 0. Ketika petugas memilih angka 0, perulangan berhenti dan perintah input.close() menutup objek Scanner yang digunakan untuk menerima masukan dari keyboard. Tanda kurung kurawal penutup menandai akhir blok perulangan, method utama, dan class MainPerpustakaan.

2.3.2 Penjelasan DataPeminjaman.java

```
package sistemperpustakaan;
import java.util.Scanner;
```

Perintah package sistemperpustakaan; digunakan untuk menunjukkan bahwa class berada dalam package sistemperpustakaan, sedangkan import java.util.Scanner; digunakan untuk mengimpor class Scanner agar program dapat menerima input dari pengguna melalui keyboard.

```
public class DataPeminjaman {

    static String[] kategoriBuku = {

        “Pemrograman”,

        “Matematika”,
```

```
    "Jaringan Komputer",
    "Basis Data"
};
static String[] hari = {
    "Senin", "Selasa", "Rabu", "Kamis",
    "Jumat", "Sabtu", "Minggu"
};
```

Deklarasi kategoriBuku menggunakan Array 1 Dimensi bertipe String untuk menyimpan empat kategori buku, yaitu Pemrograman, Matematika, Jaringan Komputer, dan Basis Data. Sementara itu, variabel hari menggunakan Array 1 Dimensi bertipe String untuk menyimpan nama hari selama satu minggu, mulai dari Senin hingga Minggu. Kedua array tersebut digunakan sebagai acuan dalam menampilkan kategori buku dan mencatat jumlah peminjaman berdasarkan hari.

```
static int[][] jumlahPeminjaman = {
    {5, 3, 4, 6, 2, 1, 0},
    {2, 4, 3, 5, 1, 0, 0},
    {3, 2, 5, 4, 3, 1, 0},
    {4, 3, 2, 5, 4, 2, 0}
};
```

Deklarasi jumlahPeminjaman menggunakan Array 2 Dimensi bertipe int untuk menyimpan data jumlah peminjaman buku berdasarkan kategori dan hari. Array ini terdiri dari empat baris yang mewakili kategori buku dan tujuh kolom yang mewakili hari dalam satu minggu. Angka-angka di dalamnya merupakan data awal peminjaman yang dapat ditampilkan, diperbarui, dan dijumlahkan sesuai kebutuhan pengelolaan perpustakaan.

```
public static void tampilKategori() {
    System.out.println("\n=== DAFTAR KATEGORI BUKU ===");
    for (int i = 0; i < kategoriBuku.length; i++) {
        System.out.println((i + 1) + ". " + kategoriBuku[i]);
    }
}
```

Method tampilKategori() digunakan untuk menampilkan daftar kategori buku yang tersimpan dalam array kategoriBuku. Perulangan for digunakan untuk membaca setiap elemen array, sedangkan kategoriBuku.length menentukan jumlah kategori yang ditampilkan. Perintah System.out.println() menampilkan nomor urut dan nama kategori buku pada layar, dengan indeks i + 1 agar penomoran dimulai dari 1.

```
public static void tampilPeminjaman() {
    System.out.println("\n=== DATA PEMINJAMAN MINGGUAN ===");
    System.out.printf("%-20s" "Kategori");
    for (String namaHari : hari) {
        System.out.printf("%-9s", namaHari);
    }
    System.out.println();
    for (int i = 0; i < kategoriBuku.length; i++) {
        System.out.printf("%-20s", kategoriBuku[i]);
        for (int j = 0; j < hari.length; j++) {
            System.out.printf("%-9d", jumlahPeminjaman[i][j]);
```

```
        }
        System.out.println();
    }
}
```

Method tampilPeminjaman() digunakan untuk menampilkan data jumlah peminjaman buku selama satu minggu dalam bentuk tabel. Perintah System.out.printf() mengatur lebar kolom agar nama kategori, nama hari, dan jumlah peminjaman tersusun rapi. Perulangan for-each digunakan untuk menampilkan seluruh nama hari, sedangkan dua perulangan for digunakan untuk membaca setiap kategori dan jumlah peminjamannya dari Array 2 Dimensi jumlahPeminjaman. Setiap baris menampilkan satu kategori buku beserta jumlah peminjaman pada setiap hari.

```
public static void ubahPeminjaman(Scanner input) {
    System.out.println("\n=== UBAH DATA PEMINJAMAN ===");
    try {
        tampilKategori();
        System.out.print("Pilih nomor kategori: ");
        int kategori = Integer.parseInt(
            input.nextLine().trim());
        if (kategori < 1 || kategori > kategoriBuku.length) {
            System.out.println("ERROR: Kategori tidak valid!");
            return;
        }
        System.out.println("\n=== PILIH HARI ===");
```

```
for (int i = 0; i < hari.length; i++) {
    System.out.println((i + 1) + ". " + hari[i]);
}
System.out.print("Pilih nomor hari: ");
int nomorHari = Integer.parseInt(
        input.nextLine().trim());
if (nomorHari < 1 || nomorHari > hari.length) {
    System.out.println("ERROR: Hari tidak valid!");
    return;
}
System.out.print("Masukkan jumlah peminjaman baru: ");
int jumlah = Integer.parseInt(
        input.nextLine().trim());
if (jumlah < 0) {
    System.out.println(
        "ERROR: Jumlah tidak boleh negatif!");
    return;
}
jumlahPeminjaman[kategori - 1][nomorHari - 1] = jumlah;
System.out.println(
    "Data peminjaman berhasil diperbarui!");
```

```
    } catch (NumberFormatException e) {
        System.out.println(
            "ERROR: Input harus berupa angka bulat!");
    }
}
```

Method ubahPeminjaman() digunakan untuk mengubah jumlah peminjaman buku berdasarkan kategori dan hari yang dipilih. Perintah tampilKategori() menampilkan daftar kategori, sedangkan perulangan for menampilkan pilihan hari. Input pengguna dikonversi menjadi bilangan bulat menggunakan Integer.parseInt(), kemudian diperiksa melalui percabangan if untuk memastikan kategori dan hari valid serta jumlah peminjaman tidak negatif. Nilai baru disimpan ke dalam Array 2 Dimensi jumlahPeminjaman, sedangkan blok try-catch menangani kesalahan apabila pengguna memasukkan input yang bukan berupa angka bulat.

```
public static void totalPeminjaman() {
    System.out.println("\n=== TOTAL PEMINJAMAN PER KATEGORI ===");
    for (int i = 0; i < kategoriBuku.length; i++) {
        int total = 0;
        for (int j = 0; j < hari.length; j++) {
            total += jumlahPeminjaman[i][j];
        }
        System.out.println(kategoriBuku[i] + ": " +
                total + " peminjaman");
    }
```

```
}
```

Method totalPeminjaman() digunakan untuk menghitung dan menampilkan total peminjaman buku pada setiap kategori selama satu minggu. Perulangan for pertama digunakan untuk membaca setiap kategori buku, sedangkan variabel total diinisialisasi dengan nilai 0 sebagai penampung hasil penjumlahan. Perulangan for kedua menjumlahkan data peminjaman setiap hari dari Array 2 Dimensi jumlahPeminjaman. Hasil penjumlahan kemudian ditampilkan bersama nama kategori buku menggunakan System.out.println().

```
public static void catatPeminjaman(int kategori, int nomorHari) {
        jumlahPeminjaman[kategori - 1][nomorHari - 1]++;
        System.out.println("Data peminjaman berhasil diperbarui.");
    }
```

Method catatPeminjaman() digunakan untuk menambahkan jumlah peminjaman pada kategori dan hari yang telah ditentukan. Parameter kategori dan nomorHari menentukan posisi data dalam Array 2 Dimensi jumlahPeminjaman. Operator ++ menambah nilai peminjaman sebanyak satu, sedangkan System.out.println() menampilkan pesan bahwa data peminjaman berhasil diperbarui.

```
public static void batalkanPeminjaman(int kategori, int nomorHari) {
        if (jumlahPeminjaman[kategori - 1][nomorHari - 1] > 0) {
            jumlahPeminjaman[kategori - 1][nomorHari - 1]--;
        }
        System.out.println("Data peminjaman berhasil disesuaikan.");
    }
```

Method batalkanPeminjaman() digunakan untuk mengurangi jumlah peminjaman pada kategori dan hari yang sesuai ketika transaksi dibatalkan. Kondisi if memastikan

bahwa jumlah peminjaman lebih dari 0 sebelum nilainya dikurangi satu menggunakan operator --, sehingga jumlah tidak menjadi negatif. Setelah proses selesai, System.out.println() menampilkan pesan bahwa data peminjaman berhasil disesuaikan.

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    int pilihan = -1;
    do {
        System.out.println("\n=== MENU DATA PEMINJAMAN ===");
        System.out.println("1. Tampilkan Kategori");
        System.out.println("2. Tampilkan Data Mingguan");
        System.out.println("3. Ubah Jumlah Peminjaman");
        System.out.println("4. Total Peminjaman");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu: ");
```

Method utama pada class ini digunakan untuk menjalankan menu pengujian data peminjaman secara mandiri. Objek Scanner digunakan untuk menerima input dari pengguna, sedangkan variabel pilihan menyimpan nomor menu dengan nilai awal -1. Perulangan do-while memastikan menu ditampilkan setidaknya satu kali. Perintah System.out.println() menampilkan pilihan untuk melihat kategori, menampilkan data peminjaman mingguan, mengubah jumlah peminjaman, menghitung total peminjaman, atau keluar dari menu.

```
if (!input.hasNextInt()) {
            System.out.println("Masukkan angka yang valid!");
            input.next();
```

```
            continue;
        }
```

Kondisi if (!input.hasNextInt()) digunakan untuk memeriksa apakah input pengguna bukan berupa bilangan bulat. Jika input tidak valid, program menampilkan pesan kesalahan, kemudian input.next() membaca dan membuang input yang salah. Perintah continue digunakan untuk melewati proses selanjutnya dan mengulang menu agar pengguna dapat memasukkan pilihan yang benar.

```
pilihan = input.nextInt();
        switch (pilihan) {
            case 1:
                tampilKategori();
                break;
            case 2:
                tampilPeminjaman();
                break;
            case 3:
                ubahPeminjaman(input);
                break;
            case 4:
                totalPeminjaman();
                break;
            case 0:
```

```
            System.out.println("Keluar dari menu peminjaman.");
            break;
        default:
            System.out.println("Pilihan tidak tersedia!");
    }
```

Variabel pilihan digunakan untuk menyimpan angka menu yang dimasukkan pengguna, kemudian percabangan switch-case menentukan fitur yang dijalankan sesuai pilihan tersebut. Setiap case memanggil fungsi untuk menampilkan kategori, melihat data peminjaman mingguan, mengubah jumlah peminjaman, atau menghitung total peminjaman. Pilihan 0 digunakan untuk keluar dari menu, sedangkan default menampilkan pesan kesalahan jika nomor menu tidak tersedia. Perintah break menghentikan proses pada setiap pilihan agar tidak berlanjut ke pilihan berikutnya.

```
} while (pilihan != 0);
        input.close();
    }
}
```

Kondisi while (pilihan != 0) digunakan untuk mengulang menu selama pengguna belum memilih angka 0. Ketika angka 0 dipilih, perulangan berhenti dan input.close() menutup objek Scanner yang digunakan untuk menerima masukan. Tanda kurung kurawal penutup menandai akhir method utama dan class DataPeminjaman.

2.3.1 Penjelasan ManajemenBuku.java

```
package sistemperpustakaan;

import java.util.ArrayList;
import java.util.Scanner;
import java.util.HashSet;
```

```
import java.util.Set;
import java.util.Locale;
```

Perintah package sistemperpustakaan; menunjukkan bahwa class berada dalam package sistemperpustakaan. Sementara itu, import java.util.ArrayList, Scanner, HashSet, Set, dan Locale digunakan untuk mengimpor class yang mendukung pengelolaan daftar buku, penerimaan input pengguna, penyimpanan data buku yang sedang dipinjam tanpa duplikasi, serta penyeragaman teks agar pencarian judul buku tidak membedakan huruf besar dan kecil.

```
public class ManajemenBuku {

    static String[][][] lokasiBuku = new String[2][3][4];

    static ArrayList<String> daftarBuku = new ArrayList<>();

    static Set<String> bukuDipinjam = new HashSet<>();
```

Deklarasi lokasiBuku menggunakan Array 3 Dimensi bertipe String untuk menyimpan lokasi buku berdasarkan 2 lantai, 3 rak pada setiap lantai, dan 4 posisi pada setiap rak. Variabel daftarBuku menggunakan ArrayList untuk menyimpan koleksi buku yang dapat ditambah atau dihapus secara dinamis. Sementara itu, bukuDipinjam menggunakan Set dengan implementasi HashSet untuk mencatat judul buku yang sedang dipinjam agar judul yang sama tidak tercatat berulang kali.

```
static {

        lokasiBuku[0][0][0] = “Java Dasar”;

        lokasiBuku[0][0][1] = “Algoritma”;

        lokasiBuku[0][1][0] = “Kalkulus”;

        lokasiBuku[0][1][1] = “Aljabar”;

        lokasiBuku[0][2][0] = “Jaringan Dasar”;
```

```
        lokasiBuku[1][0][0] = “MySQL”;

        lokasiBuku[1][0][1] = “SQL Dasar”;

        lokasiBuku[1][1][0] = “Struktur Data”;

        lokasiBuku[1][2][0] = “Sistem Operasi”;
```

Blok static digunakan untuk mengisi data awal buku ke dalam Array 3 Dimensi lokasiBuku. Setiap judul buku ditempatkan berdasarkan indeks lantai, rak, dan posisi tertentu, seperti buku Java Dasar pada lantai 1, rak 1, posisi 1. Data awal ini memudahkan petugas mengetahui lokasi buku yang tersedia sejak program dijalankan.

```
for (int i = 0; i < lokasiBuku.length; i++) {
            for (int j = 0; j < lokasiBuku[i].length; j++) {
                for (int k = 0; k < lokasiBuku[i][j].length; k++) {
                    if (lokasiBuku[i][j][k] != null) {
                        daftarBuku.add(lokasiBuku[i][j][k]);
                    }
                }
            }
        }
    }
```

Tiga perulangan for digunakan untuk menelusuri seluruh elemen Array 3 Dimensi lokasiBuku berdasarkan lantai, rak, dan posisi buku. Kondisi if (lokasiBuku[i][j][k] != null) memeriksa apakah suatu lokasi telah terisi buku. Jika terisi, judul buku ditambahkan ke dalam ArrayList daftarBuku menggunakan add(), sehingga seluruh buku yang tersimpan pada lokasi awal juga tercatat dalam daftar koleksi.

```
private static String kunciBuku(String judul) {
    return judul.trim().toLowerCase(Locale.ROOT);
}
```

Method kunciBuku() digunakan untuk menyeragamkan judul buku sebelum diproses, seperti saat memeriksa ketersediaan buku. Perintah trim() menghapus spasi di awal dan akhir judul, sedangkan toLowerCase(Locale.ROOT) mengubah seluruh huruf menjadi huruf kecil agar perbandingan judul buku tidak terpengaruh oleh perbedaan huruf besar dan kecil.

```
public static void tampilLokasi() {
    System.out.println("\n=== LOKASI PENYIMPANAN BUKU ===");
    for (int i = 0; i < lokasiBuku.length; i++) {
        System.out.println("\nLantai " + (i + 1));
        for (int j = 0; j < lokasiBuku[i].length; j++) {
            System.out.println("  Rak " + (j + 1));
            for (int k = 0; k < lokasiBuku[i][j].length; k++)
                String buku = lokasiBuku[i][j][k];
                if (buku == null) {
                    System.out.println("    Posisi " + (k + 1)
                            + ": [Kosong]");
                } else {
                    String status = bukuDipinjam.contains(
                            kunciBuku(buku))
```

```
                    ? “Sedang Dipinjam”
                    : “Tersedia”;
                System.out.println(“    Posisi “ + (k + 1)
                    + “: “ + buku
                    + “ [“ + status + “]”);
            }
        }
    }
}
```

Method tampilLokasi() digunakan untuk menampilkan lokasi penyimpanan buku berdasarkan lantai, rak, dan posisi. Tiga perulangan for digunakan untuk menelusuri setiap elemen Array 3 Dimensi lokasiBuku. Kondisi if-else memeriksa apakah posisi kosong atau terisi buku. Jika terisi, program memeriksa bukuDipinjam menggunakan contains() untuk menentukan status buku, yaitu Sedang Dipinjam atau Tersedia, kemudian menampilkan informasi tersebut pada layar.

```
public static void tambahBuku(Scanner input) {
    System.out.println(“\n=== TAMBAH BUKU ===”);
    System.out.print(“Masukkan judul buku: “);
    String judul = input.nextLine().trim();
    if (judul.isEmpty()) {
        System.out.println(“ERROR: Judul tidak boleh kosong!”);
```

```
            return;


        for (String buku : daftarBuku) {
            if (buku.equalsIgnoreCase(judul)) {
                System.out.println("ERROR: Buku sudah terdaftar!");
                return;
            }
        }
```

Method tambahBuku() digunakan untuk menambahkan buku baru ke dalam koleksi perpustakaan. Input judul buku dibaca melalui Scanner, kemudian trim() menghapus spasi di awal dan akhir judul. Kondisi if memeriksa apakah judul kosong dan menghentikan proses menggunakan return jika tidak valid. Perulangan for-each memeriksa apakah judul sudah terdaftar, sedangkan equalsIgnoreCase() membandingkan judul tanpa membedakan huruf besar dan kecil untuk mencegah duplikasi buku.

```
try {
            System.out.println("\n=== PILIH LANTAI ===");
            System.out.println("1. Lantai 1");
            System.out.println("2. Lantai 2");
            System.out.print("Pilih lantai (1-2): ");
            int lantai = Integer.parseInt(
                    input.nextLine().trim());
```

```
        if (lantai < 1 || lantai > lokasiBuku.length) {
            System.out.println("ERROR: Lantai tidak valid!");
            return;
        }
```

Blok try digunakan untuk menjalankan proses pemilihan lantai penyimpanan buku. Program menampilkan pilihan lantai 1 dan 2, kemudian membaca input pengguna dan mengubahnya menjadi bilangan bulat menggunakan Integer.parseInt(). Kondisi if memeriksa apakah nomor lantai berada dalam rentang yang tersedia, sedangkan return menghentikan proses penambahan buku jika lantai yang dipilih tidak valid.

```
System.out.println("\n=== PILIH RAK ===")
        for (int i = 0; i < lokasiBuku[lantai - 1].length; i++) {
            System.out.println((i + 1) + ". Rak " + (i + 1));
        }
        System.out.print("Pilih rak: ");
        int rak = Integer.parseInt(
                input.nextLine().trim());
        if (rak < 1 || rak > lokasiBuku[lantai - 1].length) {
            System.out.println("ERROR: Rak tidak valid!");
            return;
        }
```

Perulangan for digunakan untuk menampilkan daftar rak yang tersedia pada lantai yang telah dipilih. Input nomor rak dibaca melalui Scanner dan diubah menjadi bilangan bulat menggunakan Integer.parseInt(). Kondisi if memeriksa apakah nomor rak berada

dalam rentang yang tersedia pada lantai tersebut. Jika nomor rak tidak valid, program menampilkan pesan kesalahan dan menghentikan proses penambahan buku menggunakan return.

```
System.out.println(“\n=== PILIH POSISI ===”);
        for (int i = 0;
            i < lokasiBuku[lantai - 1][rak - 1].length;
            i++) {
           String isi = lokasiBuku[lantai - 1][rak - 1][i];
           System.out.println((i + 1) + “. Posisi “
                + (i + 1) + “ – “
                + (isi == null ? “Kosong” : “Terisi: “ + isi));
        }
        System.out.print”Pilih posisi: “);
        int posisi = Integer.parseInt(
            input.nextLine().trim());
        if (posisi < 1 ||
            posisi > lokasiBuku[lantai - 1][rak - 1].length) {
           System.out.println(“ERROR: Posisi tidak valid!”);
           return;
        }
        if (lokasiBuku[lantai - 1][rak - 1][posisi - 1]
            != null) {
```

```
            System.out.println(
                    "ERROR: Posisi sudah terisi buku!");
            return;
        }
```

Perulangan for digunakan untuk menampilkan seluruh posisi pada rak yang dipilih beserta statusnya, yaitu kosong atau sudah terisi buku. Input nomor posisi dibaca dan diubah menjadi bilangan bulat menggunakan Integer.parseInt(). Kondisi if pertama memeriksa apakah nomor posisi valid, sedangkan kondisi if kedua memastikan posisi tersebut belum ditempati buku. Jika posisi tidak valid atau sudah terisi, program menampilkan pesan kesalahan dan menghentikan proses penambahan buku menggunakan return.

```
lokasiBuku[lantai - 1][rak - 1][posisi - 1] = judul;
            daftarBuku.add(judul);
            System.out.println("\nBuku berhasil ditambahkan!");
            System.out.println("Judul  : " + judul);
            System.out.println("Lantai : " + lantai);
            System.out.println("Rak    : " + rak);
            System.out.println("Posisi : " + posisi);
            System.out.println("Status : Tersedia");
        } catch (NumberFormatException e) {
            System.out.println(
                    "ERROR: Input harus berupa angka bulat!");
        }
```

```
}
```

Perintah lokasiBuku[lantai - 1][rak - 1][posisi - 1] = judul; digunakan untuk menyimpan judul buku pada lokasi yang telah dipilih, sedangkan daftarBuku.add(judul) menambahkan judul tersebut ke dalam ArrayList koleksi buku. Selanjutnya, System.out.println() menampilkan informasi keberhasilan penambahan buku, termasuk judul, lantai, rak, posisi, dan status ketersediaannya. Blok catch (NumberFormatException e) menangani kesalahan apabila pengguna memasukkan input yang bukan berupa angka bulat saat memilih lokasi penyimpanan.

```
public static void tampilBuku() {
    System.out.println("\n=== DAFTAR KOLEKSI BUKU ===");
    if (daftarBuku.isEmpty()) {
        System.out.println("Belum ada koleksi buku.");
        return;
    }
    for (int i = 0; i < daftarBuku.size(); i++) {
        String buku = daftarBuku.get(i);
        String status = bukuDipinjam.contains(kunciBuku(buku))
                ? "Sedang Dipinjam"
                : "Tersedia";

        System.out.println((i + 1) + ". " + buku
                +" [" + status + "]");
    }
```

```
    System.out.println("Total koleksi: " + daftarBuku.size());
}
```

Method tampilBuku() digunakan untuk menampilkan seluruh koleksi buku yang tersimpan dalam ArrayList daftarBuku. Kondisi if dengan isEmpty() memeriksa apakah daftar buku kosong, sedangkan perulangan for digunakan untuk membaca dan menampilkan setiap judul buku beserta statusnya. Method kunciBuku() dan contains() digunakan untuk memeriksa apakah buku sedang dipinjam atau masih tersedia. Pada akhir proses, daftarBuku.size() digunakan untuk menampilkan jumlah seluruh koleksi buku.

```
public static void cariBuku(Scanner input) {
    System.out.println("\n=== CARI BUKU ===");

    System.out.print("Masukkan judul buku: ");
    String judul = input.nextLine().trim();

    if (judul.isEmpty()) {
        System.out.println("ERROR: Judul tidak boleh kosong!");
        return;
    }
    for (int i = 0; i < lokasiBuku.length; i++) {
        for (int j = 0; j < lokasiBuku[i].length; j++) {
            for (int k = 0; k < lokasiBuku[i][j].length; k++) {
                String buku = lokasiBuku[i][j][k];
```

```
            if (buku != null &&
                buku.equalsIgnoreCase(judul)) {

                String status = bukuDipinjam.contains(
                    kunciBuku(buku))
                    ? "Sedang Dipinjam"
                    : "Tersedia";

                System.out.println("\nBuku ditemukan!");
                System.out.println("Judul  : " + buku);
                System.out.println("Lantai : " + (i + 1));
                System.out.println("Rak    : " + (j + 1));
                System.out.println("Posisi : " + (k + 1));
                System.out.println("Status : " + status);
                return;
            }
        }
    }
}
System.out.println("Buku tidak ditemukan!");
}
```

Method cariBuku() digunakan untuk mencari buku berdasarkan judul yang dimasukkan pengguna. Input judul diperiksa menggunakan isEmpty() untuk memastikan tidak kosong. Tiga perulangan for digunakan untuk menelusuri seluruh lokasi buku dalam Array 3 Dimensi, sedangkan equalsIgnoreCase() membandingkan judul tanpa membedakan huruf besar dan kecil. Jika buku ditemukan, program menampilkan judul, lokasi penyimpanan, dan status ketersediaannya. Jika tidak ditemukan setelah seluruh lokasi diperiksa, program menampilkan pesan bahwa buku tidak ditemukan.

```
public static void hapusBuku(Scanner input) {
    System.out.println("\n=== HAPUS BUKU ===");
    System.out.print("Masukkan judul buku: ");
    String judul = input.nextLine().trim();
    if (judul.isEmpty()) {
        System.out.println("ERROR: Judul tidak boleh kosong!");
        return;
    }
    for (int i = 0; i < lokasiBuku.length; i++) {
        for (int j = 0; j < lokasiBuku[i].length; j++) {
            for (int k = 0; k < lokasiBuku[i][j].length; k++) {
                String buku = lokasiBuku[i][j][k];


                if (buku != null &&
                        buku.equalsIgnoreCase(judul)) {
                    if (bukuDipinjam.contains(
```

```
                    kunciBuku(buku))) {
                System.out.println(
                    "ERROR: Buku sedang dipinjam!");
                System.out.println(
                    "Buku tidak dapat dihapus.");
                return;
            }
            lokasiBuku[i][j][k] = null;
            daftarBuku.remove(buku);
            System.out.println(
                "Buku berhasil dihapus!");
            return;
        }
    }
  }
  System.out.println("Buku tidak ditemukan!");
}
```

Method hapusBuku() digunakan untuk menghapus buku dari koleksi perpustakaan berdasarkan judul yang dimasukkan pengguna. Kondisi if memeriksa apakah judul kosong, sedangkan tiga perulangan for digunakan untuk mencari buku pada seluruh lokasi penyimpanan. Jika buku ditemukan, program memeriksa status peminjaman melalui

bukuDipinjam.contains() agar buku yang sedang dipinjam tidak dapat dihapus. Jika buku tidak sedang dipinjam, lokasi penyimpanannya diatur menjadi null dan judulnya dihapus dari ArrayList menggunakan daftarBuku.remove(). Apabila buku tidak ditemukan atau sedang dipinjam, program menampilkan pesan yang sesuai.

```
public static boolean pinjamBuku(String judul) {
    if (judul == null || judul.trim().isEmpty()) {
        System.out.println("ERROR: Judul buku tidak valid!");
        return false;
    }
    String kunci = kunciBuku(judul);
    boolean ditemukan = false;
    for (String buku : daftarBuku) {
        if (kunciBuku(buku).equals(kunci)) {
            ditemukan = true;
            break;
        }
    }
    if (!ditemukan) {
        System.out.println("ERROR: Buku tidak ditemukan");
        return false;
    }
    if (bukuDipinjam.contains(kunci)) {
```

```
            System.out.println(
                "ERROR: Buku sedang dipinjam mahasiswa lain!");
            return false;
        }
        bukuDipinjam.add(kunci);
        System.out.println(
            "Buku tersedia dan berhasil dipinjam.");
        return true;
    }
```

Method pinjamBuku() digunakan untuk memproses peminjaman buku dengan memeriksa validitas judul, keberadaan buku dalam koleksi, dan status peminjamannya. Kondisi if memastikan judul tidak kosong, sedangkan perulangan for-each mencari buku dalam daftarBuku dengan membandingkan judul yang telah diseragamkan menggunakan kunciBuku(). Jika buku tidak ditemukan atau sedang dipinjam, proses dihentikan dengan mengembalikan nilai false. Jika buku tersedia, judulnya ditambahkan ke bukuDipinjam menggunakan add(), kemudian program menampilkan pesan keberhasilan dan mengembalikan nilai true.

```
public static void kembalikanBuku(String judul) {
        if (judul == null || judul.trim().isEmpty()) {
            System.out.println("ERROR: Judul buku tidak valid!");
            return;
        }
```

```
    String kunci = kunciBuku(judul);
    if (bukuDipinjam.remove(kunci)) {
        System.out.println(
            "Status buku berhasil dikembalikan menjadi tersedia.");
    } else {
        System.out.println(
            "Buku tidak sedang dalam status dipinjam.");
    }
}
```

Method kembalikanBuku() digunakan untuk mengubah status buku yang sedang dipinjam menjadi tersedia kembali. Kondisi if memeriksa apakah judul buku valid, kemudian kunciBuku() menyeragamkan judul agar pencocokan tidak terpengaruh oleh spasi atau huruf besar dan kecil. Perintah bukuDipinjam.remove(kunci) menghapus judul dari daftar buku yang sedang dipinjam. Jika penghapusan berhasil, program menampilkan pesan bahwa buku telah tersedia kembali; jika tidak, program memberi tahu bahwa buku tidak sedang dalam status dipinjam.

```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    int pilihan = -1;
    do {
        System.out.println("\n=== MENU MANAJEMEN BUKU ===");
        System.out.println("1. Tampilkan Lokasi Buku");
        System.out.println("2. Tambah Buku");
```

```
System.out.println("3. Tampilkan Daftar Buku");
System.out.println("4. Cari Buku");
System.out.println("5. Hapus Buku");
System.out.println("0. Keluar");
System.out.print("Pilih menu:");
try {
   pilihan = Integer.parseInt(
         input.nextLine().trim());
   switch (pilihan) {
      case 1:
         tampilLokasi();
         break;
      case 2:
         tambahBuku(input);
         break;
      case 3:
         tampilBuku();
         break
      case 4:
         cariBuku(input);
         break
```

```
                case 5:
                    hapusBuku(input);
                    break;
                case 0:
                    System.out.println(
                            "Keluar dari manajemen buku");
                    break;
                default:
                    System.out.println(
                            "ERROR: Menu tidak tersedia!");
                }

            } catch (NumberFormatException e) {
                System.out.println(
                        "ERROR: Input harus berupa angka bulat!");
            }
        } while (pilihan != 0);
        input.close();
    }
}
```

Method utama pada class ManajemenBuku digunakan untuk menjalankan menu pengelolaan buku secara mandiri. Objek Scanner menerima input dari pengguna, sedangkan variabel pilihan menyimpan nomor menu. Perulangan do-while menampilkan menu secara berulang, dan Integer.parseInt() mengubah input menjadi bilangan bulat. Percabangan switch-case menjalankan fungsi sesuai pilihan, seperti menampilkan lokasi, menambah, menampilkan, mencari, dan menghapus buku. Blok try-catch menangani input yang bukan berupa angka bulat, sedangkan perulangan berhenti ketika pengguna memilih angka 0 dan objek Scanner ditutup menggunakan input.close().

2.3.1 Penjelasan TransaksiAntrean.java

```
package sistemperpustakaan;

import java.util.Queue;

import java.util.LinkedList;

import java.util.Stack;

import java.util.Scanner;
```

Perintah package sistemperpustakaan; menunjukkan bahwa class berada dalam package sistemperpustakaan. Sementara itu, import java.util.Queue, LinkedList, Stack, dan Scanner digunakan untuk mengimpor struktur data antrean FIFO, implementasi antrean menggunakan LinkedList, struktur data tumpukan LIFO untuk menyimpan riwayat transaksi, serta class Scanner untuk menerima input dari pengguna melalui keyboard.

```
public class TransaksiAntrean {

    static Queue<String> antrean = new LinkedList<>();

    static class Transaksi {

        String nama;

        String judul;
```

```
    int kategori;
    int hari;
    Transaksi(String nama, String judul,
            int kategori, int hari) {
        this.nama = nama;
        this.judul = judul;
        this.kategori = kategori;
        this.hari = hari;
    }
```

Deklarasi antrean menggunakan struktur data Queue dengan implementasi LinkedList untuk menyimpan nama mahasiswa yang menunggu giliran peminjaman buku berdasarkan prinsip FIFO (First In, First Out). Class Transaksi digunakan untuk menyimpan informasi setiap transaksi, meliputi nama mahasiswa, judul buku, kategori, dan hari peminjaman. Konstruktor Transaksi berfungsi mengisi atribut tersebut berdasarkan data yang diterima saat objek transaksi dibuat.

```
@Override
    public String toString() {
        return nama + " meminjam " + judul
                + " ("
                + DataPeminjaman.kategoriBuku[kategori - 1]
                + ", "
                + DataPeminjaman.hari[hari - 1]
```

```
            + ")";
    }
}
static Stack<Transaksi> riwayat = new Stack<>();
```

Method toString() digunakan untuk mengubah informasi transaksi menjadi teks yang menampilkan nama mahasiswa, judul buku, kategori, dan hari peminjaman. Data kategori dan hari diambil dari array kategoriBuku dan hari pada class DataPeminjaman, dengan indeks dikurangi satu agar sesuai dengan penomoran yang dimulai dari 1. Sementara itu, variabel riwayat menggunakan struktur data Stack untuk menyimpan objek transaksi berdasarkan prinsip LIFO (Last In, First Out), sehingga transaksi terakhir dapat diakses atau dibatalkan terlebih dahulu.

```
public static void tambahAntrean(Scanner input) {
    System.out.println("\n=== TAMBAH ANTREAN ===");
    System.out.print("Masukkan nama mahasiswa: ");
    String nama = input.nextLine().trim();
    if (nama.isEmpty()) {
        System.out.println("ERROR: Nama tidak boleh kosong!");
        return;
    }
    antrean.offer(nama);

    System.out.println(nama + " berhasil masuk antrean.");
    System.out.println("Jumlah antrean: " + antrean.size());
```

```
}
```

Method tambahAntrean() digunakan untuk menambahkan mahasiswa ke dalam antrean peminjaman buku. Nama mahasiswa dibaca melalui Scanner, kemudian trim() menghapus spasi di awal dan akhir nama. Kondisi if memeriksa apakah nama kosong dan menghentikan proses jika tidak valid. Perintah antrean.offer(nama) menambahkan mahasiswa ke antrean berdasarkan prinsip FIFO (First In, First Out), sedangkan antrean.size() digunakan untuk menampilkan jumlah mahasiswa yang sedang mengantre.

```
public static void tampilAntrean() {
    System.out.println("\n=== DAFTAR ANTREAN ===");
    if (antrean.isEmpty()) {
        System.out.println("Antrean masih kosong.");
        return;
    }
    int nomor = 1;
    for (String nama : antrean) {
        System.out.println(nomor + ". " + nama);
        nomor++;
    }
}
```

Method tampilAntrean() digunakan untuk menampilkan daftar mahasiswa yang sedang mengantre untuk meminjam buku. Kondisi if dengan isEmpty() memeriksa apakah antrean kosong, sedangkan variabel nomor digunakan untuk memberikan nomor urut kepada setiap mahasiswa. Perulangan for-each menampilkan nama mahasiswa sesuai

urutan kedatangan, sehingga penerapan prinsip FIFO (First In, First Out) dapat terlihat pada daftar antrean.

```
public static void layaniAntrean(Scanner input) {
    System.out.println("\n=== LAYANI ANTREAN ===");
    if (antrean.isEmpty()) {
        System.out.println("ERROR: Antrean kosong!");
        return;
    }
    String nama = antrean.peek();
    System.out.println("Mahasiswa: " + nama);
    System.out.print("Masukkan judul buku: ");
    String judul = input.nextLine().trim();
    if (judul.isEmpty()) {
        System.out.println("ERROR: Judul tidak boleh kosong!");
        return;
    }
    try {
        DataPeminjaman.tampilKategori();
        System.out.print("Pilih kategori (1-4): ");
        int kategori = Integer.parseInt(
            input.nextLine().trim());
```

```
if (kategori < 1 ||
    kategori > DataPeminjaman.kategoriBuku.length) {
    System.out.println("ERROR: Kategori tidak valid!");
    return;
}
System.out.println("\n=== PILIH HARI ===");
for (int i = 0; i < DataPeminjaman.hari.length; i++) {
    System.out.println((i + 1) + ". "
            + DataPeminjaman.hari[i]);
}
System.out.print("Pilih hari (1-7): ");
int hari = Integer.parseInt(
        input.nextLine().trim());
if (hari < 1 || hari > DataPeminjaman.hari.length) {
    System.out.println("ERROR: Hari tidak valid!");
    return;
}
if (!ManajemenBuku.pinjamBuku(judul)) {
    System.out.println("Peminjaman gagal!");
    return;
}
```

```
            Transaksi transaksi =
                    new Transaksi(nama, judul, kategori, hari);
            riwayat.push(transaksi);
            DataPeminjaman.catatPeminjaman(kategori, hari);
            antrean.poll();
            System.out.println("\nPEMINJAMAN BERHASIL!");
            System.out.println("Mahasiswa : " + nama);
            System.out.println("Judul Buku: " + judul);
            System.out.println("Kategori  : "
                    + DataPeminjaman.kategoriBuku[kategori - 1]);
            System.out.println("Hari      : "
                    + DataPeminjaman.hari[hari - 1]);
        } catch (NumberFormatException e) {
            System.out.println(
                    "ERROR: Input harus berupa angka bulat!");
        }
    }
```

Method layaniAntrean() digunakan untuk memproses peminjaman buku oleh mahasiswa yang berada di urutan pertama antrean. Program memeriksa apakah antrean kosong, membaca judul buku, serta memvalidasi kategori dan hari peminjaman. Jika buku tersedia, data transaksi disimpan ke dalam Stack menggunakan push(), jumlah peminjaman diperbarui melalui catatPeminjaman(), dan mahasiswa dikeluarkan dari

antrean menggunakan poll(). Blok try-catch menangani input yang bukan berupa angka bulat, sedangkan program menampilkan rincian peminjaman setelah transaksi berhasil.

```
public static void tampilRiwayat() {
    System.out.println("\n=== RIWAYAT TRANSAKSI ===");
    if (riwayat.isEmpty()) {
        System.out.println("Belum ada transaksi.");
        return;
    }
    for (int i = riwayat.size() - 1; i >= 0; i--) {
        System.out.println((riwayat.size() - i)
            + ". " + riwayat.get(i));
    }
}
```

Method tampilRiwayat() digunakan untuk menampilkan seluruh riwayat transaksi peminjaman buku. Kondisi if dengan isEmpty() memeriksa apakah Stack riwayat masih kosong. Jika terdapat transaksi, perulangan for menampilkan data mulai dari indeks terakhir hingga indeks pertama, sehingga transaksi terbaru ditampilkan terlebih dahulu sesuai prinsip LIFO (Last In, First Out). Setiap transaksi ditampilkan bersama nomor urut dan informasi mahasiswa, judul buku, kategori, serta hari peminjaman.

```
public static void transaksiTerakhir() {
    System.out.println("\n=== TRANSAKSI TERAKHIR ===");
    if (riwayat.isEmpty()) {
        System.out.println("Belum ada transaksi.");
```

```
        return;
    }
    System.out.println(riwayat.peek());
}
```

Method transaksiTerakhir() digunakan untuk menampilkan transaksi peminjaman yang paling baru. Kondisi if dengan isEmpty() memeriksa apakah Stack riwayat kosong. Jika belum ada transaksi, program menampilkan pesan bahwa transaksi belum tersedia. Jika terdapat transaksi, perintah riwayat.peek() mengambil dan menampilkan transaksi teratas tanpa menghapusnya dari Stack, sesuai prinsip LIFO (Last In, First Out).

```
public static void batalkanTransaksi() {
    System.out.println("\n=== BATALKAN TRANSAKSI ===");
    if (riwayat.isEmpty()) {
        System.out.println("ERROR: Tidak ada transaksi!");
        return;
    }
    Transaksi transaksi = riwayat.pop();
    ManajemenBuku.kembalikanBuku(transaksi.judul);
    DataPeminjaman.batalkanPeminjaman(
        transaksi.kategori, transaksi.hari);
    System.out.println("\nTransaksi berhasil dibatalkan!");
    System.out.println("Mahasiswa : " + transaksi.nama);
    System.out.println("Judul Buku: " + transaksi.judul);
```

```
}
```

Method batalkanTransaksi() digunakan untuk membatalkan transaksi peminjaman yang paling baru. Kondisi if memeriksa apakah Stack riwayat kosong, sedangkan riwayat.pop() mengambil sekaligus menghapus transaksi teratas sesuai prinsip LIFO (Last In, First Out). Selanjutnya, ManajemenBuku.kembalikanBuku() mengubah status buku menjadi tersedia dan DataPeminjaman.batalkanPeminjaman() mengurangi jumlah peminjaman sesuai kategori dan hari transaksi. Setelah proses selesai, program menampilkan informasi bahwa transaksi berhasil dibatalkan beserta nama mahasiswa dan judul buku.

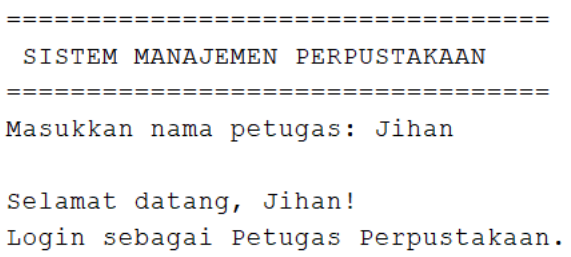
```
public static void main(String[] args) {
    Scanner input = new Scanner(System.in);
    int pilihan = -1;
    do {
        System.out.println("\n=== MENU TRANSAKSI ===');
        System.out.println("1. Tambah Antrean");
        System.out.println("2. Tampilkan Antrean");
        System.out.println("3. Layani Antrean");
        System.out.println("4. Tampilkan Riwayat");
        System.out.println("5. Lihat Transaksi Terakhir");
        System.out.println("6. Batalkan Transaksi");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu: ");
        try {
```

```
pilihan = Integer.parseInt(
        input.nextLine().trim());
switch (pilihan) {
    case 1:
        tambahAntrean(input);
        break;
    case 2:
        tampilAntrean();
        break;
    case 3:
        layaniAntrean(input);
        break;
    case 4:
        tampilRiwayat();
        break;
    case 5:
        transaksiTerakhir();
        break;
    case 6:
        batalkanTransaksi();
        break;
```

```
                case 0:
                    System.out.println("Keluar.");
                    break;
                default:
                    System.out.println("Menu tidak tersedia!");
            }
        } catch (NumberFormatException e) {
            System.out.println("ERROR: Input harus angka!");
        }
    } while (pilihan != 0);
    input.close();
  }
}
```

Method utama pada class TransaksiAntrean digunakan untuk menjalankan menu pengelolaan antrean dan transaksi secara mandiri. Objek Scanner digunakan untuk menerima input, sedangkan variabel pilihan menyimpan nomor menu yang dipilih pengguna. Perulangan do-while menampilkan menu secara berulang, dan percabangan switch-case menjalankan fitur sesuai pilihan, seperti menambah antrean, melayani peminjaman, menampilkan riwayat, serta membatalkan transaksi. Blok try-catch menangani input yang bukan berupa angka bulat, sedangkan perulangan berhenti ketika pengguna memilih angka 0 dan objek Scanner ditutup menggunakan input.close().

2.4 Tampilan dan Hasil Eksekusi Program



Gambar 2. 1 Tampilan Awal dan Input Nama Petugas

Gambar tersebut menunjukkan tampilan awal program Sistem Manajemen Perpustakaan Kampus. Petugas diminta memasukkan nama sebelum dapat mengakses menu utama. Setelah nama dimasukkan, program menampilkan pesan selamat datang dan konfirmasi bahwa pengguna masuk sebagai petugas perpustakaan.



Gambar 2. 2 Tampilan Menu Utama Sistem Manajemen Perpustakaan Kampus

Menu utama Sistem Manajemen Perpustakaan Kampus menyediakan 15 pilihan fitur dan satu pilihan untuk keluar dari program. Fitur yang tersedia meliputi pengelolaan kategori dan koleksi buku, pencatatan peminjaman, pengaturan lokasi penyimpanan, pengelolaan antrean mahasiswa, serta pencatatan dan pembatalan transaksi. Petugas dapat memilih fitur dengan memasukkan nomor menu sesuai kebutuhan.

```
Pilih menu (0-15): 1

=== DAFTAR KATEGORI BUKU ===
1. Pemrograman
2. Matematika
3. Jaringan Komputer
4. Basis Data
```

Gambar 2. 3 Tampilan Daftar Kategori Buku pada Sistem Manajemen Perpustakaan Kampus

Program menampilkan daftar empat kategori buku, yaitu Pemrograman, Matematika, Jaringan Komputer, dan Basis Data. Daftar ini digunakan sebagai acuan bagi petugas untuk mengelompokkan buku dan mencatat jumlah peminjaman berdasarkan kategori yang dipilih.

```
Pilih menu (0-15): 2

=== DATA PEMINJAMAN MINGGUAN ===
Kategori            Senin    Selasa   Rabu     Kamis    Jumat    Sabtu    Minggu
Pemrograman         5        3        4        6        2        1        0
Matematika          2        4        3        5        1        0        0
Jaringan Komputer   3        2        5        4        3        1        0
Basis Data          4        3        2        5        4        2        0
```

Gambar 2. 4 Tampilan Data Peminjaman Buku Mingguan Berdasarkan Kategori dan Hari

Program menampilkan data peminjaman buku selama satu minggu berdasarkan kategori, yaitu Pemrograman, Matematika, Jaringan Komputer, dan Basis Data. Data disusun dalam bentuk tabel dengan nama hari sebagai kolom dan jumlah peminjaman sebagai nilai pada setiap kategori. Tampilan ini membantu petugas memantau serta membandingkan jumlah peminjaman buku setiap hari.

```
Pilih menu (0-15): 3

=== UBAH DATA PEMINJAMAN ===

=== DAFTAR KATEGORI BUKU ===
1. Pemrograman
2. Matematika
3. Jaringan Komputer
4. Basis Data
Pilih nomor kategori: 1

=== PILIH HARI ===
1. Senin
2. Selasa
3. Rabu
4. Kamis
5. Jumat
6. Sabtu
7. Minggu
Pilih nomor hari: 2
Masukkan jumlah peminjaman baru: 7
Data peminjaman berhasil diperbarui!
```

Gambar 2. 5 Tampilan Proses Perubahan Data Peminjaman Buku

Program menyediakan fitur untuk mengubah data peminjaman berdasarkan kategori buku dan hari yang dipilih. Pada contoh ini, petugas memilih kategori Pemrograman dan hari Selasa, kemudian memasukkan jumlah peminjaman baru sebanyak 7. Program memvalidasi input dan menampilkan pesan bahwa data peminjaman berhasil diperbarui.

```
Pilih menu (0-15): 4

=== TOTAL PEMINJAMAN PER KATEGORI ===
Pemrograman: 25 peminjaman
Matematika: 15 peminjaman
Jaringan Komputer: 18 peminjaman
Basis Data: 20 peminjaman
```

Gambar 2. 6 Tampilan Total Peminjaman Buku Berdasarkan Kategori

Program menampilkan total peminjaman buku untuk setiap kategori berdasarkan penjumlahan data peminjaman selama satu minggu. Hasil menunjukkan bahwa kategori Pemrograman memiliki 25 peminjaman, Matematika 15 peminjaman, Jaringan Komputer 18 peminjaman, dan Basis Data 20 peminjaman. Informasi ini membantu petugas mengetahui jumlah peminjaman pada setiap kategori buku.

```
Pilih menu (0-15): 5

=== LOKASI PENYIMPANAN BUKU ===

Lantai 1
  Rak 1
    Posisi 1: Java Dasar [Tersedia]
    Posisi 2: Algoritma [Tersedia]
    Posisi 3: [Kosong]
    Posisi 4: [Kosong]
  Rak 2
    Posisi 1: Kalkulus [Tersedia]
    Posisi 2: Aljabar [Tersedia]
    Posisi 3: [Kosong]
    Posisi 4: [Kosong]
  Rak 3
    Posisi 1: Jaringan Dasar [Tersedia]
    Posisi 2: [Kosong]
    Posisi 3: [Kosong]
    Posisi 4: [Kosong]
```

Gambar 2. 7 Tampilan Lokasi Penyimpanan dan Status Ketersediaan Buku

```
Lantai 2
  Rak 1
    Posisi 1: MySQL [Tersedia]
    Posisi 2: SQL Dasar [Tersedia]
    Posisi 3: [Kosong]
    Posisi 4: [Kosong]
  Rak 2
    Posisi 1: Struktur Data [Tersedia]
    Posisi 2: [Kosong]
    Posisi 3: [Kosong]
    Posisi 4: [Kosong]
  Rak 3
    Posisi 1: Sistem Operasi [Tersedia]
    Posisi 2: [Kosong]
    Posisi 3: [Kosong]
    Posisi 4: [Kosong]
```

Gambar 2. 8 Tampilan Lokasi Penyimpanan dan Status Ketersediaan Buku Lantai 2

Program menampilkan lokasi penyimpanan buku berdasarkan lantai, rak, dan posisi. Setiap posisi menunjukkan judul buku beserta status ketersediaannya, yaitu Tersedia atau Kosong. Informasi ini membantu petugas mengetahui tempat penyimpanan buku dan menentukan lokasi yang masih dapat digunakan untuk menyimpan koleksi baru.

```
Pilih menu (0-15): 6

=== TAMBAH BUKU ===
Masukkan judul buku: Database

=== PILIH LANTAI ===
1. Lantai 1
2. Lantai 2
Pilih lantai (1-2): 1

=== PILIH RAK ===
1. Rak 1
2. Rak 2
3. Rak 3
Pilih rak: 3

=== PILIH POSISI ===
1. Posisi 1 - Terisi: Jaringan Dasar
2. Posisi 2 - Kosong
3. Posisi 3 - Kosong
4. Posisi 4 - Kosong
Pilih posisi: 2
```

Gambar 2. 9 Tampilan Proses Pemilihan Lokasi Penyimpanan Buku

Program menampilkan proses penambahan buku dengan meminta petugas memasukkan judul buku, memilih lantai penyimpanan, menentukan rak, dan memilih posisi yang masih kosong. Pada contoh ini, petugas memasukkan judul buku Database, kemudian memilih lantai 1, rak 3, dan posisi 2 sebagai lokasi penyimpanan.

```
Buku berhasil ditambahkan!
Judul  : Database
Lantai : 1
Rak    : 3
Posisi : 2
Status : Tersedia
```

Gambar 2. 10. Tampilan Konfirmasi Penambahan Buku Berhasil

Program menampilkan konfirmasi bahwa buku berjudul Database berhasil ditambahkan ke dalam koleksi perpustakaan. Informasi yang ditampilkan meliputi judul buku, lantai, rak, posisi penyimpanan, dan status ketersediaan buku. Status Tersedia menunjukkan bahwa buku telah tercatat dan dapat dipinjam oleh mahasiswa.

```
Pilih menu (0-15): 7

=== DAFTAR KOLEKSI BUKU ===
1. Java Dasar [Tersedia]
2. Algoritma [Tersedia]
3. Kalkulus [Tersedia]
4. Aljabar [Tersedia]
5. Jaringan Dasar [Tersedia]
6. MySQL [Tersedia]
7. SQL Dasar [Tersedia]
8. Struktur Data [Tersedia]
9. Sistem Operasi [Tersedia]
10. Database [Tersedia]
Total koleksi: 10
```

Gambar 2. 11 Tampilan Daftar Koleksi Buku dan Status Ketersediaan Mahasiswa

Program menampilkan seluruh koleksi buku yang tersimpan dalam perpustakaan beserta status ketersediaannya. Setiap buku diberi nomor urut untuk memudahkan identifikasi, sedangkan status Tersedia menunjukkan bahwa buku dapat dipinjam. Pada tampilan tersebut, terdapat 10 buku yang tercatat dalam koleksi, termasuk buku Database yang baru ditambahkan.

```
Pilih menu (0-15): 8

=== CARI BUKU ===
Masukkan judul buku: MySQL

Buku ditemukan!
Judul  : MySQL
Lantai : 2
Rak    : 1
Posisi : 1
Status : Tersedia
```

Gambar 2. 12 Tampilan Hasil Pencarian Buku dan Informasi Lokasi Penyimpanan

Program menyediakan fitur pencarian buku berdasarkan judul yang dimasukkan petugas. Pada contoh ini, buku berjudul MySQL berhasil ditemukan dan program menampilkan informasi lokasi penyimpanannya, yaitu lantai 2, rak 1, posisi 1. Status Tersedia menunjukkan bahwa buku tersebut dapat dipinjam oleh mahasiswa.

```
Pilih menu (0-15): 9

=== HAPUS BUKU ===
Masukkan judul buku: Database
Buku berhasil dihapus!
```

Gambar 2. 13 Tampilan Proses Penghapusan Buku dari Koleksi Perpustakaan

Program menyediakan fitur untuk menghapus buku berdasarkan judul yang dimasukkan petugas. Pada contoh ini, petugas memasukkan judul Database, kemudian program memproses penghapusan dan menampilkan pesan bahwa buku berhasil dihapus. Fitur ini membantu petugas mengelola koleksi agar data buku yang tidak diperlukan dapat dihapus dari daftar dan lokasi penyimpanannya.

```
Pilih menu (0-15): 10

=== TAMBAH ANTREAN ===
Masukkan nama mahasiswa: Jihan
Jihan berhasil masuk antrean.
Jumlah antrean: 1
```

Gambar 2. 14 Tampilan Proses Penambahan Mahasiswa ke Antrean Peminjaman Buku

Program menyediakan fitur untuk menambahkan mahasiswa ke dalam antrean peminjaman buku. Pada contoh ini, mahasiswa bernama Jihan berhasil dimasukkan ke dalam antrean, dan program menampilkan jumlah antrean sebanyak satu orang. Proses ini menerapkan prinsip FIFO (First In, First Out), yaitu mahasiswa yang masuk lebih dahulu akan dilayani lebih dahulu.

```
Pilih menu (0-15): 11

=== DAFTAR ANTREAN ===
1. Jihan
```

Gambar 2. 15 Tampilan Daftar Antrean Mahasiswa Peminjaman Buku

Program menampilkan daftar mahasiswa yang sedang mengantre untuk meminjam buku. Pada contoh ini, terdapat satu mahasiswa bernama Jihan yang berada di urutan pertama antrean. Daftar tersebut menunjukkan urutan kedatangan mahasiswa sesuai prinsip FIFO (First In, First Out), sehingga mahasiswa yang datang lebih dahulu akan dilayani terlebih dahulu.

```
Pilih menu (0-15): 12

=== LAYANI ANTREAN ===
Mahasiswa: Jihan
Masukkan judul buku: MySQL

=== DAFTAR KATEGORI BUKU ===
1. Pemrograman
2. Matematika
3. Jaringan Komputer
4. Basis Data
Pilih kategori (1-4): 4

=== PILIH HARI ===
1. Senin
2. Selasa
3. Rabu
4. Kamis
5. Jumat
6. Sabtu
7. Minggu
Pilih hari (1-7): 1
Buku tersedia dan berhasil dipinjam.
Data peminjaman berhasil diperbarui.
```

Gambar 2. 16 Tampilan Proses Pelayanan Antrean dan Input Data Peminjaman Buku

Program menampilkan proses pelayanan antrean mahasiswa bernama Jihan. Petugas memasukkan judul buku MySQL, memilih kategori Basis Data, dan menentukan hari Senin sebagai hari peminjaman. Program memeriksa ketersediaan buku dan memperbarui data peminjaman setelah buku berhasil dipinjam.

```
PEMINJAMAN BERHASIL!
Mahasiswa : Jihan
Judul Buku: MySQL
Kategori  : Basis Data
Hari      : Senin
```

Gambar 2. 17 Tampilan Konfirmasi Keberhasilan Transaksi Peminjaman Buku

Program menampilkan konfirmasi bahwa peminjaman buku berhasil dilakukan oleh mahasiswa bernama Jihan. Informasi yang ditampilkan meliputi nama mahasiswa, judul buku MySQL, kategori Basis Data, dan hari Senin. Hasil ini menunjukkan bahwa proses peminjaman berhasil diproses dan dicatat oleh sistem.

```
Pilih menu (0-15): 13

=== RIWAYAT TRANSAKSI ===
1. Jihan meminjam MySQL (Basis Data, Senin)
```

Gambar 2. 18 Tampilan Riwayat Transaksi Peminjaman Buku

Program menampilkan riwayat transaksi peminjaman yang telah dilakukan. Pada contoh ini, tercatat bahwa Jihan meminjam buku MySQL dari kategori Basis Data pada hari Senin. Riwayat ini digunakan untuk membantu petugas memeriksa transaksi yang telah tersimpan dalam sistem.

```
Pilih menu (0-15): 14

=== TRANSAKSI TERAKHIR ===
Jihan meminjam MySQL (Basis Data, Senin)
```

Gambar 2. 19 Tampilan Informasi Transaksi Peminjaman Terakhir

Program menampilkan transaksi peminjaman yang paling baru tercatat dalam sistem. Pada contoh ini, transaksi terakhir menunjukkan bahwa Jihan meminjam buku MySQL dari kategori Basis Data pada hari Senin. Fitur ini memudahkan petugas memeriksa transaksi terbaru tanpa harus menampilkan seluruh riwayat peminjaman.

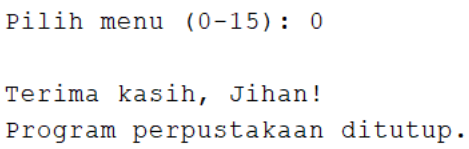
```
Pilih menu (0-15): 15

=== BATALKAN TRANSAKSI ===
Status buku berhasil dikembalikan menjadi tersedia.
Data peminjaman berhasil disesuaikan.

Transaksi berhasil dibatalkan!
Mahasiswa : Jihan
Judul Buku: MySQL
```

Gambar 2. 20 Tampilan Proses Pembatalan Transaksi Peminjaman Buku

Program menyediakan fitur untuk membatalkan transaksi peminjaman terakhir. Pada contoh ini, transaksi peminjaman buku MySQL oleh Jihan berhasil dibatalkan. Sistem mengubah status buku menjadi tersedia kembali dan menyesuaikan data jumlah peminjaman sesuai transaksi yang dibatalkan. Proses ini menerapkan prinsip LIFO (Last In, First Out), yaitu transaksi yang terakhir dilakukan akan dibatalkan terlebih dahulu.



Gambar 2. 21 Tampilan Penutupan Program Sistem Manajemen Perpustakaan Kampus

Program menampilkan pesan penutupan setelah petugas memilih menu 0. Sistem memberikan ucapan terima kasih dengan menyertakan nama petugas, kemudian menampilkan informasi bahwa program perpustakaan telah ditutup. Proses ini menandakan bahwa penggunaan program telah berakhir.

BAB III
PENGUJIAN DAN PEMBAHASAN

3.1 Hasil Pengujian Program

Tabel 3. 1 Hasil Pengujian Program

Fitur yang Diuji	Hasil yang Diharapkan	Hasil
Array 1D	Kategori buku dan nama hari dapat ditampilkan	Berhasil
Array 2D	Data peminjaman mingguan dapat ditampilkan dan diperbarui	Berhasil
Array 3D	Lokasi buku berdasarkan lantai, rak, dan posisi dapat ditampilkan	Berhasil
ArrayList	Buku dapat ditambah, ditampilkan, dicari, dan dihapus	Berhasil
Queue (FIFO)	Mahasiswa masuk antrean dan dilayani sesuai urutan kedatangan	Berhasil
Stack (LIFO)	Riwayat transaksi dapat ditampilkan dan transaksi terakhir dapat dibatalkan	Berhasil
Peminjaman buku	Buku yang tersedia dapat dipinjam dan statusnya diperbarui	Berhasil

Berdasarkan hasil pengujian, program Sistem Manajemen Perpustakaan Kampus telah mampu menjalankan fungsi utama, seperti menampilkan kategori buku, mengelola data peminjaman, mengatur lokasi penyimpanan, serta menambah, mencari, dan menghapus koleksi buku. Fitur antrean mahasiswa dan riwayat transaksi juga telah diterapkan menggunakan struktur data Queue dan Stack. Proses peminjaman dan pembatalan transaksi dapat memperbarui status ketersediaan buku serta jumlah peminjaman. Pengujian lebih lanjut

diperlukan untuk memastikan urutan pelayanan antrean dan penanganan input yang tidak valid berjalan sesuai dengan yang diharapkan.

3.2 Analisis Fungsionalitas Program

Program Sistem Manajemen Perpustakaan Kampus dirancang untuk membantu petugas dalam mengelola koleksi buku dan proses peminjaman secara terstruktur. Penerapan enam struktur data memungkinkan setiap fitur menjalankan fungsi yang berbeda sesuai kebutuhan sistem. Array 1 Dimensi digunakan untuk menyimpan kategori buku dan nama hari, sedangkan Array 2 Dimensi digunakan untuk mencatat jumlah peminjaman berdasarkan kategori dan hari. Array 3 Dimensi membantu mengatur lokasi penyimpanan buku berdasarkan lantai, rak, dan posisi, sementara ArrayList digunakan untuk mengelola koleksi buku secara dinamis.

Struktur data Queue diterapkan untuk mengatur antrean mahasiswa berdasarkan urutan kedatangan menggunakan prinsip FIFO (First In, First Out). Sementara itu, Stack digunakan untuk menyimpan riwayat transaksi dengan prinsip LIFO (Last In, First Out), sehingga transaksi terakhir dapat dilihat dan dibatalkan terlebih dahulu. Program juga dilengkapi validasi input dan error handling untuk menangani kesalahan masukan, seperti nama kosong atau pilihan menu yang bukan berupa angka. Secara keseluruhan, integrasi keenam struktur data tersebut mendukung pengelolaan perpustakaan kampus melalui menu interaktif yang mudah digunakan oleh petugas.

BAB IV
PENUTUP

4.1 Kesimpulan

Program Sistem Manajemen Perpustakaan Kampus berhasil dirancang menggunakan bahasa pemrograman Java dengan menerapkan enam struktur data, yaitu Array 1 Dimensi, Array 2 Dimensi, Array 3 Dimensi, ArrayList, Stack, dan Queue. Setiap struktur data memiliki fungsi yang saling mendukung, mulai dari menyimpan kategori buku dan data peminjaman, mengatur lokasi penyimpanan, mengelola koleksi buku, hingga memproses antrean mahasiswa dan riwayat transaksi.

Program dilengkapi dengan menu interaktif yang memungkinkan petugas melakukan penambahan, pencarian, penampilan, dan penghapusan buku, serta mengelola proses peminjaman dan pembatalan transaksi. Penerapan prinsip FIFO pada Queue dan LIFO pada Stack membantu mengatur urutan pelayanan mahasiswa dan pengelolaan transaksi terakhir. Selain itu, validasi input dan error handling diterapkan untuk menangani beberapa kesalahan masukan pengguna. Secara keseluruhan, program ini membantu memahami penerapan struktur data dalam pengembangan sistem pengelolaan perpustakaan yang terstruktur dan saling terintegrasi.